

The Evolution of AI Agents and Modern AI Engineering

Technical Research Report

1. LLM Applications

Large Language Model (LLM) applications are the foundational layer of modern AI engineering: a single request-response pipeline where a user prompt is sent to a model (e.g. GPT-4, Claude, Gemini) and a completion is returned. Early LLM applications were largely stateless — chatbots, summarizers, and Q&A tools built by prompt engineering alone. The next stage added Retrieval-Augmented Generation (RAG), where relevant documents are fetched from a vector database and injected into the context window before generation, grounding answers in external knowledge. Real-world examples include ChatGPT's basic chat mode, customer-support Q&A bots, and document-summarization tools embedded in products like Notion AI. These applications are reactive: they respond once per turn and have no ability to independently pursue a multi-step goal, which motivated the shift toward AI agents.

2. AI Agents

An AI agent extends an LLM application with autonomy: given a goal, it plans, acts, observes results, and iterates until the goal is satisfied — without a human specifying each step. The LLM becomes a reasoning “controller” wrapped in a loop of Perceive → Plan → Act → Observe. This pattern was popularized by the ReAct (Reason+Act) prompting method and operationalized in tools such as AutoGPT, GitHub Copilot's agent mode, and Claude Code / Claude Agent SDK, which power autonomous coding agents like Devin and Claude's own computer-use agents. Figure 1 contrasts the simple LLM pipeline with the agentic loop.

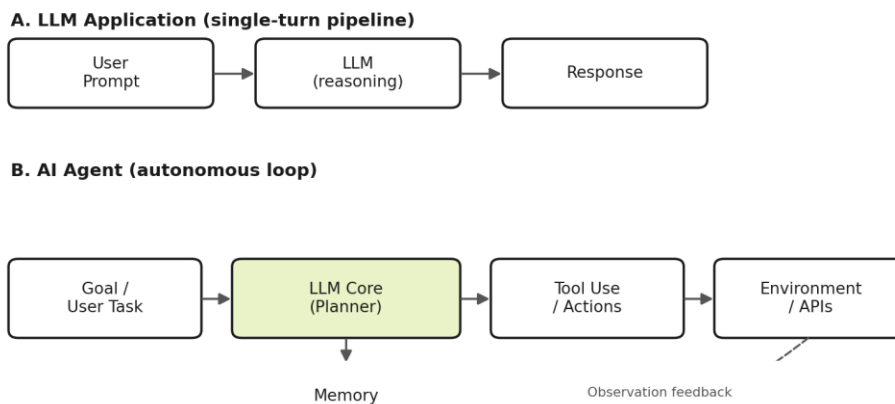


Figure 1. Linear LLM application pipeline (A) vs. an autonomous agent loop with feedback (B).

3. Tool Calling

Tool calling (a.k.a. function calling) lets an LLM invoke external functions — web search, code execution, database queries, or third-party APIs — by emitting a structured JSON call that the host application executes and feeds back as an observation. This is the mechanism that turns a language model into an actor in the real world. Anthropic and OpenAI both expose native tool-use APIs, and the Model Context Protocol (MCP), introduced by Anthropic in late 2024, has become a widely adopted open standard for connecting agents to external tools and data sources in a model-agnostic way. Real-world examples: Claude in Chrome browsing the web, Claude for Excel manipulating spreadsheets, and ChatGPT plugins/Actions calling third-party services.

4. Memory

Because LLMs are stateless between calls, agents need explicit memory. Short-term (working) memory lives in the context window — the running conversation and recent tool outputs. Long-term memory persists across sessions, typically via vector databases (Pinecone, Chroma, pgvector) for semantic recall, or structured stores for facts and preferences (as in this very product's cross-conversation memory feature). Memory allows agents to avoid repeating work, personalize behavior, and reason over information larger than the context window through retrieval. Figure 2 shows memory as a component the planner reads from and writes to on every loop iteration.

5. Planning

Planning is how an agent decomposes a high-level goal into an ordered sequence of executable steps. Approaches range from simple single-step ReAct reasoning, to explicit task-decomposition (plan-and-execute), to tree-based search (Tree-of-Thoughts) for exploring multiple solution paths. Modern coding agents like Devin and Claude Code combine planning with self-correction: after each tool call, the agent re-evaluates whether the plan still holds and revises it if a step fails. This closes the loop between planning and execution shown in Figure 2.

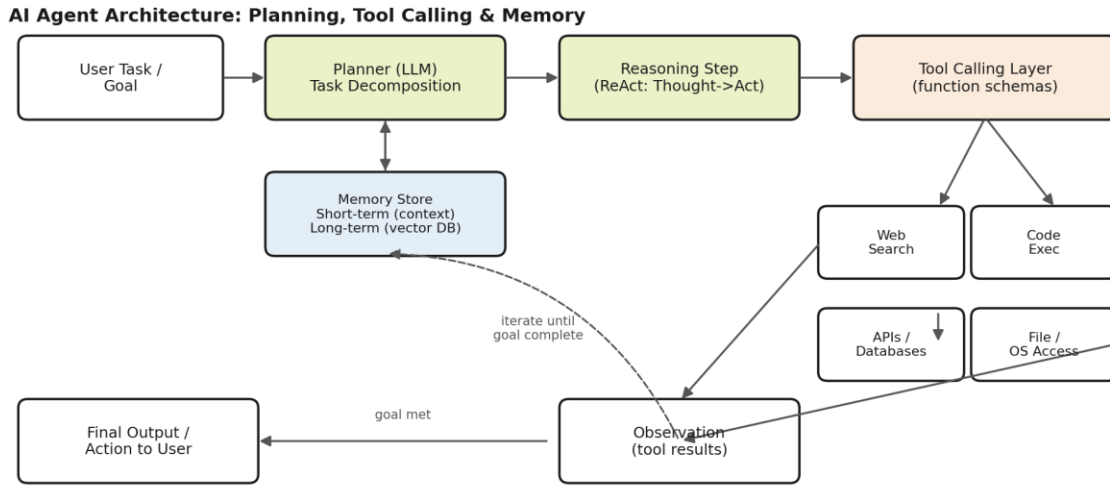


Figure 2. Detailed agent architecture showing the planner, memory store, tool-calling layer, and the observe-iterate loop.

6. Multi-Agent Systems

As tasks grow complex, a single monolithic agent becomes hard to control and debug, motivating multi-agent systems where specialized agents collaborate under an orchestrator. Common patterns include orchestrator-worker (a lead agent routes subtasks to specialist agents), debate/critique (agents check each other's work), and role-based crews. Frameworks such as LangGraph, Microsoft's Agent Framework (successor to AutoGen), CrewAI, OpenAI's Agents SDK, and Google's ADK are the dominant orchestration layers used in production in 2026, with LangGraph reporting the largest enterprise deployment footprint and CrewAI favored for fast prototyping of role-based teams. Real-world example: a software-engineering multi-agent system with a planning agent, a coding agent, and a QA/review agent working over a shared task board, mirrored in Figure 3.

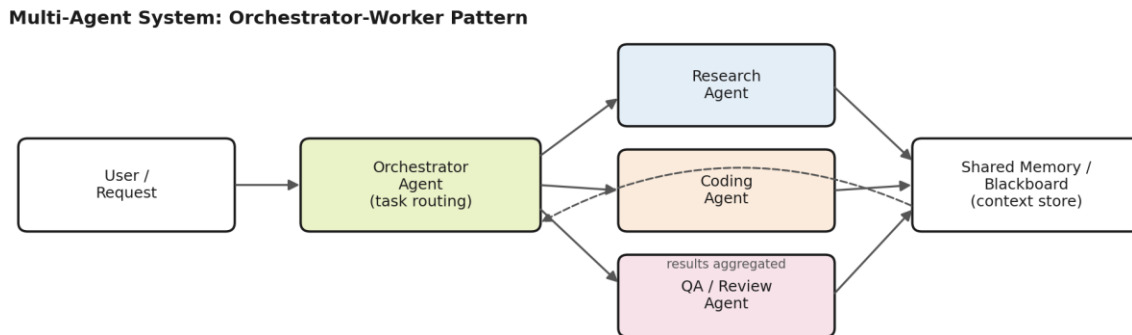


Figure 3. Orchestrator-worker multi-agent pattern with a shared memory/blackboard for coordination.

7. Future of AI Engineering

- Standardized interoperability: protocols like MCP are becoming the “USB-C for AI tools,” letting any agent connect to any tool or data source without custom glue code.
- Agent-native infrastructure: observability, evaluation, and guardrail tooling (LangSmith, tracing, sandboxed execution) is maturing as agents move from demos to production.
- Convergence toward fewer, more capable frameworks after 2024-2025's Cambrian explosion of agent libraries, as noted across current framework comparisons.
- Longer-horizon autonomy: agents increasingly execute multi-hour tasks (e.g., full coding projects) with human checkpoints rather than turn-by-turn supervision.
- AI engineering as a discipline is shifting from prompt engineering toward system design: context management, tool design, evaluation harnesses, and safety guardrails are now core skills.

References

1. Yao, S. et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” ICLR 2023.
2. Anthropic. “Introducing the Model Context Protocol.” anthropic.com/news, Nov. 2024.
3. LangChain. “The Best AI Agent Frameworks in 2026.” langchain.com/resources/ai-agent-frameworks.
4. Presenc AI. “Multi-Agent Orchestration Frameworks 2026 (LangGraph, CrewAI, AutoGen, Swarm).” presenc.ai/research, May 2026.

5. DEV Community. "LangGraph vs CrewAI vs AutoGen: The Complete Multi-Agent AI Orchestration Guide for 2026." dev.to, Jun. 2026.
6. Alice Labs. "Best AI Agent Frameworks 2026: 7 Compared." alicelabs.ai/en/insights, Jun. 2026.